Manual de Usuario: OakLand-js

Descripción General

OakLand-js es un lenguaje de programación interpretado, basado en la implementación de **gramáticas PEG** (Parsing Expression Grammar) usando la biblioteca **PEG.js**. El lenguaje soporta **structs**, acceso a propiedades, y modificación de las mismas. Además, proporciona un entorno de desarrollo donde se pueden ejecutar scripts, analizar la estructura sintáctica y generar reportes de errores y tablas de símbolos.

Estructura del Proyecto

El proyecto está compuesto por varios módulos JavaScript que se encargan de la gestión de los distintos aspectos del lenguaje. A continuación, se describe la función de cada archivo clave:

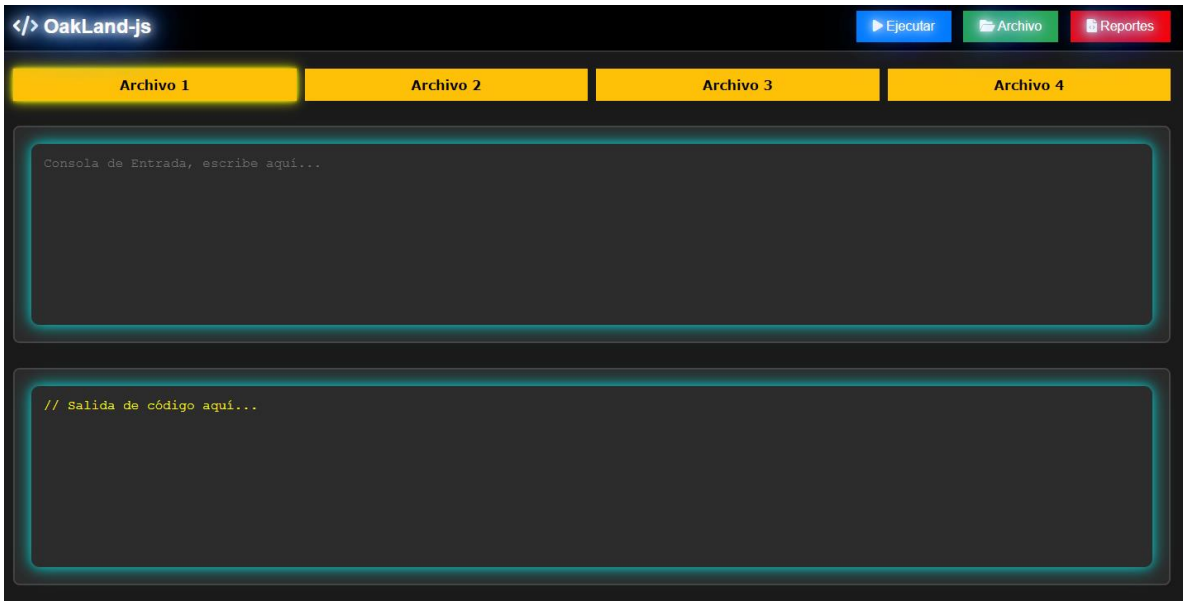
1. **clase.js**: Define la estructura y comportamiento de las clases y structs. Incluye la lógica para gestionar los métodos y propiedades de una clase.
2. **config.js**: Proporciona configuraciones globales del proyecto.
3. **embebidas.js**: Incluye funciones predefinidas y operaciones embebidas dentro del lenguaje.
4. **entorno.js**: Implementa el manejo de los entornos de ejecución (tablas de símbolos), incluyendo la declaración y almacenamiento de variables y funciones.
5. **foranea.js**: Define métodos que pueden ser invocados desde fuera de la clase, es decir, funciones externas.
6. **instancia.js**: Representa las instancias de las clases o structs, permitiendo acceder y modificar propiedades de estas instancias.
7. **interprete.js**: Implementa el patrón visitor, que recorre el árbol sintáctico y ejecuta las acciones asociadas. Aquí es donde se interpreta el código fuente.
8. **invocable.js**: Define el comportamiento para funciones invocables, incluyendo las clases y funciones.
9. **nodos.js**: Define los distintos tipos de nodos en el Árbol Sintáctico Abstracto (AST), que representan las diversas construcciones sintácticas del lenguaje.
10. **scripts.js**: Maneja la interacción entre el intérprete y la interfaz gráfica, incluyendo la ejecución de código y la generación de reportes.
11. **tool.js**: Contiene utilidades para generar el visitor a partir de los nodos y clases.
12. **transferencia.js**: Se encarga de la transferencia de información entre el intérprete y otras estructuras del proyecto.
13. **visitor.js**: Define la lógica para recorrer el AST, visitando cada nodo y ejecutando las acciones necesarias.

Uso del Editor

Interfaz Gráfica

La interfaz gráfica consta de tres secciones principales:

- **Editor de Código**: Un área donde el usuario puede escribir su código en **OakLand-js**.
- **Consola de Salida**: Muestra la salida del código ejecutado o los errores detectados.
- **Botones de Control**:
    - **Ejecutar**: Ejecuta el código escrito en el editor.
    - **Archivo**: Permite cargar un archivo de código externo.
    - **Reportes**: Genera un reporte de errores y símbolos del programa ejecutado.



Ejecución de Código

1. **Escribir Código**: El usuario puede escribir su código en el editor proporcionado.

```
struct Persona {
string nombre;
int edad;
}
Persona p = Persona {nombre: "Juan", edad: 30};
System.out.println(p.nombre);
```

2. **Ejecutar el Código**: Una vez escrito, se presiona el botón **Ejecutar** para interpretar el código. La consola de salida mostrará los resultados de la ejecución.
3. **Reporte de Errores**: Si hay errores sintácticos o semánticos, estos se muestran en la consola de salida y se pueden consultar con más detalle generando un **reporte de errores** al presionar el botón **Reportes**.

Reportes

El sistema genera dos tipos de reportes:

1. **Reporte de Símbolos**: Muestra una tabla con las variables declaradas, su tipo, valor y la ubicación en el código.
2. **Reporte de Errores**: Muestra una tabla con los errores sintácticos y semánticos detectados, incluyendo la línea y columna del error.

Ambos reportes pueden abrirse en una nueva ventana tras ejecutar el código.

Manejo de Errores

OakLand-js proporciona un sistema de manejo de errores robusto que captura:

- **Errores Sintácticos**: Errores que ocurren durante el análisis de la estructura del código, como falta de llaves o paréntesis.
- **Errores Semánticos**: Errores durante la ejecución, como el intento de acceder a una propiedad no definida.